

Project 1 Writeups

1. Dumb Classifiers

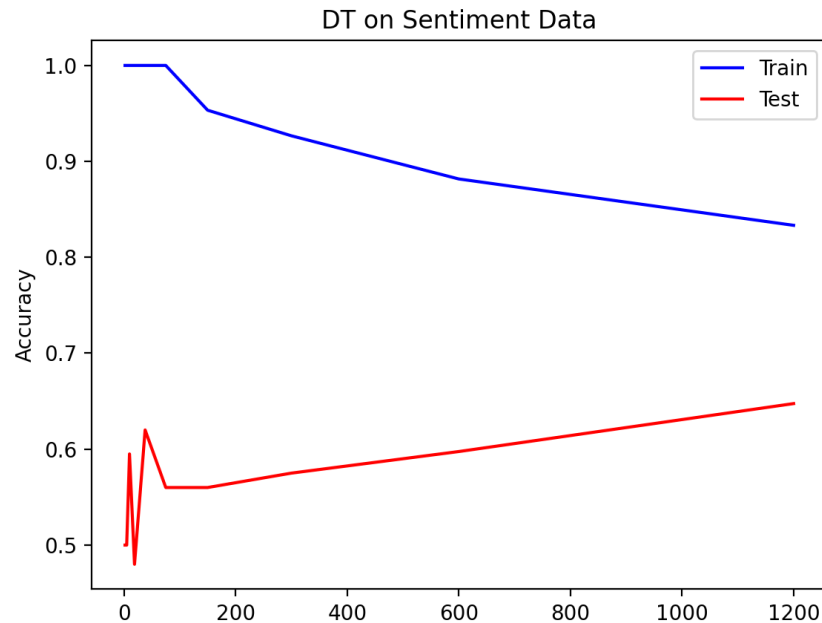
- **WU1:** why is this computation equivalent to computing classification accuracy?

```
>>> mean((datasets.TennisData.Y > 0) ==  
(h.predictAll(datasets.TennisData.X) > 0))  
0.6428571428571429
```

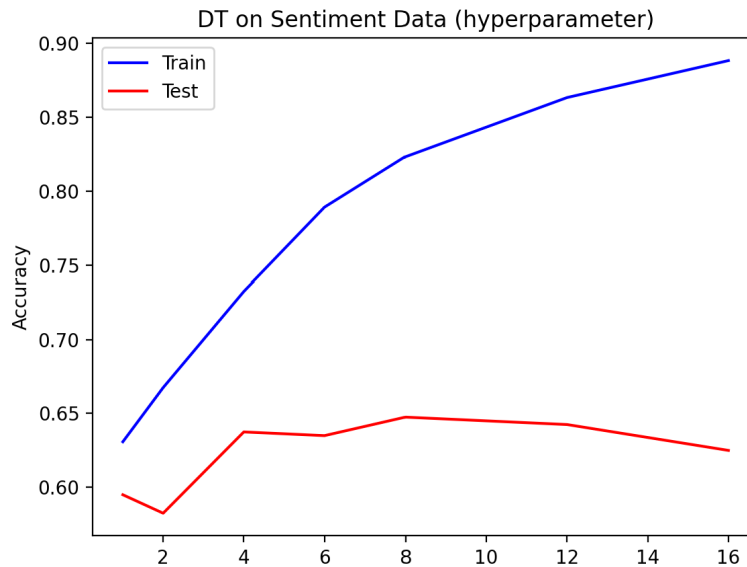
- Accuracy = (# of true positives + # of true non-positives) / (# of observations).
- (A) `datasets.TennisData.Y > 0` maps every positive class (there is only one, +1) to True and every non-positive class (there is only one, -1) to False.
- (B) `h.predictAll(datasets.TennisData.X) > 0` maps every observation to its predicted class then to True if the predicted class is positive and False if non-positive.
- This means True in either comes from the class +1 and False in either comes from the class -1.
- (C) `A == B` maps every pair of class label and class prediction to True if prediction equals label and False otherwise.
- `mean(C)` takes the arithmetic mean and treats True as 1 and False as 0, so it counts the number of times prediction equals label, positive or negative, which is (# of true positives + # of true non-positives) and divides by length, which is (# of observations).

2. Decision Tree

- **WU2:** We should see training accuracy (roughly) going down and test accuracy (roughly) going up. Why does training accuracy tend to go down? Why is test accuracy not monotonically increasing? You should also see jaggedness in the test curve toward the left. Why?



-
- As the number of points increases, the perfect (with respect to training data) decision tree T with minimum depth becomes more complex, so when T has a depth greater than maxDepth , the decision tree begins to have some leaves that include points from different classes, and the heterogeneity increases as depth of T increases as the number of points increases. This explains why training accuracy is decreasing.
- In general, as the number of points increases, the sampling variability decreases, thus the train/test split variability. This explains in part why test accuracy is increasing. Toward the left as the training distribution is less complex, the increasing complexity which captures either noise or structure in training data outweighs the decreasing variability between training and test data. This is a reason why monotonic increase is interrupted, specifically at the left.
- **WU3:** You should see training accuracy monotonically increasing and test accuracy making something like a hill. Which of these is *guaranteed* to happen and which is just something we might expect to happen? Why?



-
- Monotonic non-decrease of training accuracy is guaranteed. Intuitively, we learn more about training data and perform on the same set, so training accuracy should not decrease. A more complex decision tree builds on the simpler one since the algorithm is greedy and the data is the same, so consider a leaf of the simpler one that has $[(a + b), (c + d)]$ observations labeled $[(+1), (-1)]$ and that the next best feature splits data into leaves $[(a), (c)]$ and $[(b), (d)]$. Assume $a + b > c + d$. Suppose $a > c$. If $b \geq d$, both trees will make $c + d$ errors here, and if $b < d$, the simpler will make $c + d$ errors and the more complex will make $c + b < c + d$ errors, hence with greater depth we are guaranteed a non-decreasing change.
- We expect test accuracy to make something like a hill. We get that beyond some depth the model is overfitting training data, capturing training noise and consequently making more errors on test data. That is not guaranteed to happen. It is possible that the data has some underlying structure that is perfect such that there is no noise and sufficiently complex such that a depth greater than 16 is required to capture the whole structure and the training/test split doesn't lead to simpler structures so that 'overfitting' training data does not have a negative effect on test accuracy.

3. KNN

- **WU4:** For the digits data, generate train/test curves for varying values of K and epsilon (you figure out what are good ranges, this time). Include those

curves: do you see evidence of overfitting and underfitting? Next, using $K=5$, generate learning curves for this data.

For train/test curve:

$K = [2, 4, 6, 8, 10, 12, 14]$

$e = [0.001, 0.01, 0.1, 1, 10, 100, 1000]$

Results: Training accuracy 0.94, test accuracy 0.93 ($k = 2$)

Training accuracy 0.91, test accuracy 0.9 ($k = 4$)

Training accuracy 0.9, test accuracy 0.91 ($k = 6$)

Training accuracy 0.87, test accuracy 0.89 ($k = 8$)

Training accuracy 0.86, test accuracy 0.87 ($k = 10$)

Training accuracy 0.86, test accuracy 0.86 ($k = 12$)

Training accuracy 0.86, test accuracy 0.85 ($k = 14$)

Training accuracy 1, test accuracy 0.5 ($e = 0.001$)

Training accuracy 1, test accuracy 0.5 ($e = 0.01$)

Training accuracy 1, test accuracy 0.5 ($e = 0.1$)

Training accuracy 1, test accuracy 0.5 ($e = 1$)

Training accuracy 0.74, test accuracy 0.74 ($e = 10$)

Training accuracy 0.5, test accuracy 0.5 ($e = 100$)

Training accuracy 0.5, test accuracy 0.5 ($e = 1000$)

****** There is clear overfitting for epsilon balls where the epsilon is from 0.001 to 1.

For learning curves, $k = 5$:

Training classifier on 2 points...

Training accuracy 0.5, test accuracy 0.5

Training classifier on 4 points...

Training accuracy 0.75, test accuracy 0.5

Training classifier on 7 points...

Training accuracy 0.571429, test accuracy 0.71

Training classifier on 13 points...

Training accuracy 0.615385, test accuracy 0.81

Training classifier on 25 points...

Training accuracy 0.8, test accuracy 0.85

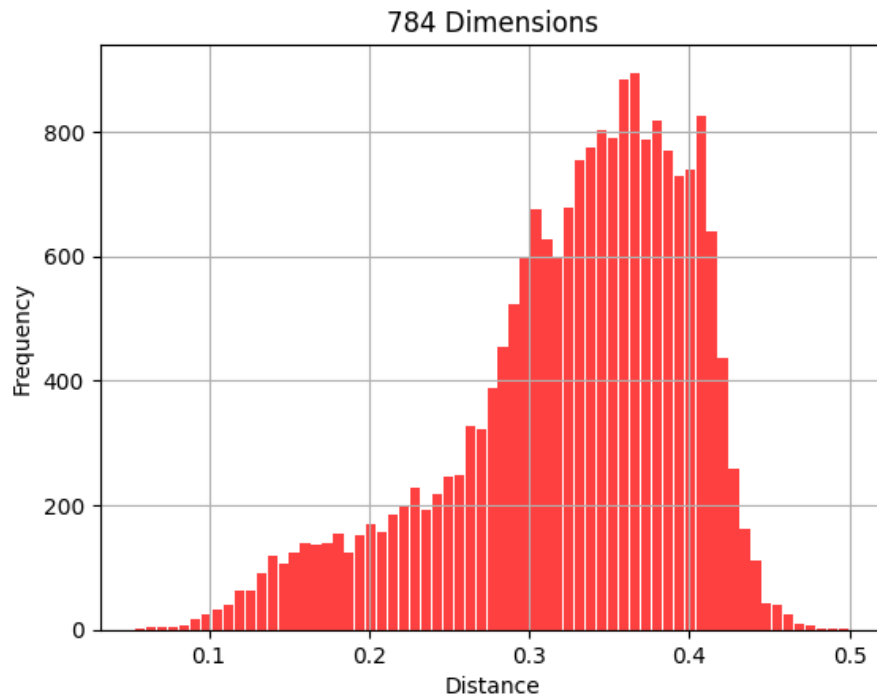
Training classifier on 50 points...

Training accuracy 0.88, test accuracy 0.86

Training classifier on 100 points...

Training accuracy 0.92, test accuracy 0.92

- WU5:
- A. First, get a histogram of the raw digits data in 784 dimensions. You'll probably want to use the `computeDistances` function together with the plotting in HighD.



- B. Rewrite `computeDistances` so that it can subsample features down to some fixed dimensionality. For example, you might write `computeDistancesSubdims(data, d)`, where `d` is the target dimensionality. In this function, you should pick `d` dimensions at random (I would suggest generating a permutation of the number `[1..784]` and then taking the first `d` of them), and then compute the distance but *only* looking at those dimensions.

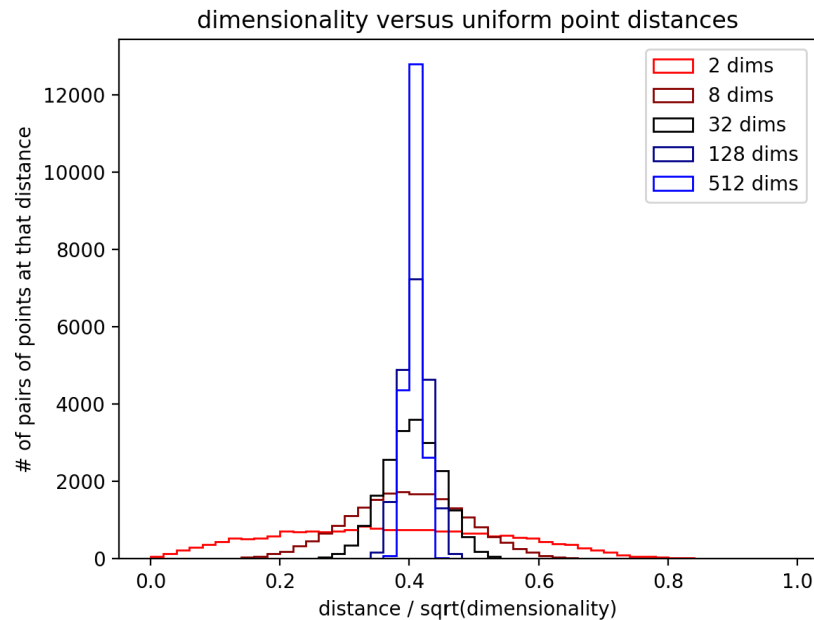
```
def computeDistancesSubdims(data, d):
    N = len(data)
    dist = []
    D = len(data[0])
    dims_to_use = random_dimensions(D, d)
    for n in range(N):
        for m in range(n):
            dist_nm = 0.0
            for dim in dims_to_use:
                dist_nm += (data[n][dim] - data[m][dim]) ** 2
            dist.append(sqrt(dist_nm) / sqrt(d))
    return dist

def random_dimensions(D, d):
    all_dims = list(range(D))
    random.shuffle(all_dims)
```

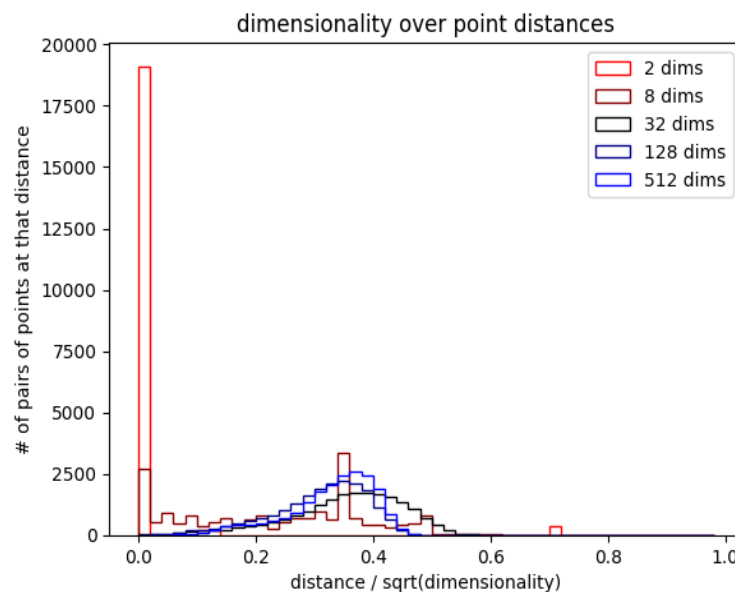
```
return all_dims[:d]
```

- C. Generate an equivalent plot to HighD with d in $[2, 8, 32, 128, 512]$ but for the digits data rather than the random data. Include a copy of both plots and describe the differences.

- Original:



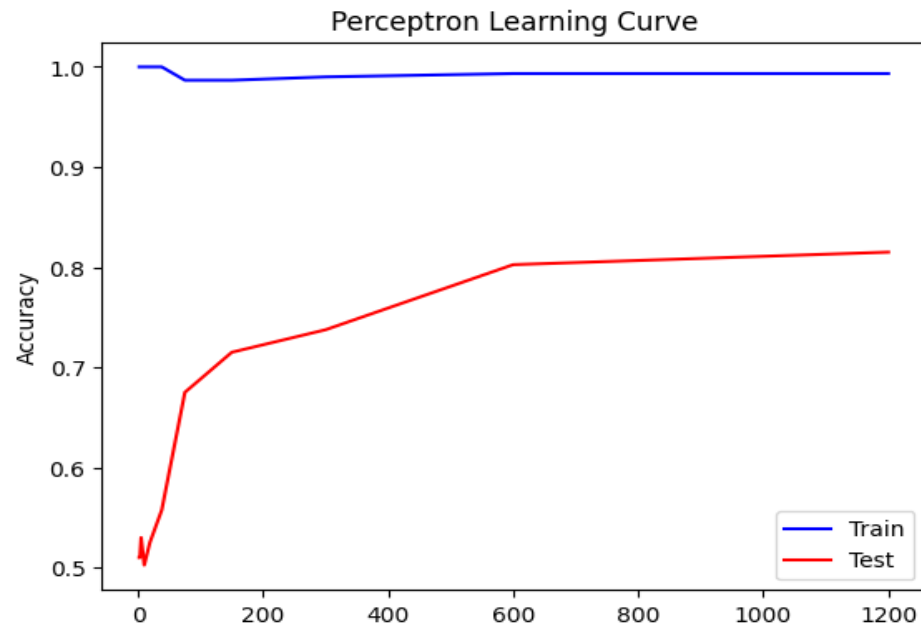
- Modified:



- There are two difference: (1) the original is symmetric whereas the modified is not and (2) the original approaches a single value as the number of dimensions increases whereas the modified approaches a single value as the number of dimensions decreases.

4. Perceptron

- **WU6:** Using the tools provided, generate (a) a learning curve (x-axis=number of training examples) for the perceptron (5 epochs) on the sentiment data and



- (b) a plot of the number of epochs versus train/test accuracy on the entire dataset.

